

Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-Trained Language Models

QUANJUN ZHANG, Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

CHUNRONG FANG, YI ZHENG, YAXIN ZHANG, and YUAN ZHAO, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

RUBING HUANG, School of Computer Science and Engineering, Macau University of Science and Technology, Taipa, China

JIANYI ZHOU, Huawei Cloud Computing Technologies Co., Ltd., Beijing, China

YUN YANG, Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia

TAO ZHENG and ZHENYU CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China and Shenzhen Research Institute of Nanjing University, Shenzhen, China

Unit testing validates the correctness of the units of the software system under test and serves as the cornerstone in improving software quality and reliability. To reduce manual efforts in writing unit tests, some techniques have been proposed to generate test assertions automatically, including Deep Learning (DL)-based, retrieval-based, and integration-based ones. Among them, recent integration-based approaches inherit from both DL-based and retrieval-based approaches and are considered state-of-the-art. Despite being promising, such integration-based approaches suffer from inherent limitations, such as retrieving assertions with lexical matching while ignoring meaningful code semantics and generating assertions with a limited training corpus.

This work is partially supported by the National Natural Science Foundation of China (U24A20337, 61932012, 62372228), Shenzhen-Hong Kong-Macau Technology Research Programme (Type C) (Grant No. SGDX20230821091559018), CCF-Huawei Populus Grove Fund (CCF-HuaweiSE202304, CCF-HuaweiSY202306), and the Fundamental Research Funds for the Central Universities (14380029).

Authors' Contact Information: **Quanjun Zhang**, Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: quanjun.zhang@smail.nju.edu.cn; **Chunrong Fang** (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: fangchunrong@nju.edu.cn; **Yi Zheng**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: 201250182@smail.nju.edu.cn; **Yaxin Zhang**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhangyaxin032@gmail.com; **Yuan Zhao**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: allen-zcrazy@gmail.com; **Rubing Huang** (corresponding author), School of Computer Science and Engineering, Macau University of Science and Technology, Taipa, China; e-mail: rbhuang@must.edu.mo; **Jianyi Zhou**, Huawei Cloud Computing Technologies Co., Ltd., Beijing, China; e-mail: zhoujianyi2@huawei.com; **Yun Yang**, Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia; e-mail: yyang@swin.edu.au; **Tao Zheng**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China and Shenzhen Research Institute of Nanjing University, Shenzhen, China; e-mail: zt@nju.edu.cn; **Zhenyu Chen** (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China and Shenzhen Research Institute of Nanjing University, Shenzhen, China; e-mail: zychen@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/8-ART209

<https://doi.org/10.1145/3721128>

In this article, we propose a novel Retrieval-Augmented Deep Assertion Generation (RetriGen) approach based on a hybrid assertion retriever and a Pre-Trained Language Model (PLM)-based assertion generator. Given a focal-test, RetriGen first builds a hybrid assertion retriever to search for the most relevant test–assert pair from external codebases. The retrieval process takes both lexical similarity and semantical similarity into account via a token-based and an embedding-based retriever, respectively. RetriGen then treats assertion generation as a sequence-to-sequence task and designs a PLM-based assertion generator to predict a correct assertion with historical test–assert pairs and the retrieved external assertion. Although our concept is general and can be adapted to various off-the-shelf encoder–decoder PLMs, we implement RetriGen to facilitate assertion generation based on the recent CodeT5 model. We conduct extensive experiments to evaluate RetriGen against six state-of-the-art approaches across two large-scale datasets and two metrics. The experimental results demonstrate that RetriGen achieves 57.66% and 73.24% in terms of accuracy and CodeBLEU, outperforming all baselines with an average improvement of 50.66% and 14.14%, respectively. Furthermore, RetriGen generates 1,598 and 1,818 unique correct assertions that all baselines fail to produce, 3.71X and 4.58X more than the most recent approach EDITAS. We also demonstrate that adopting other PLMs can provide substantial advancement, e.g., four additionally utilized PLMs outperform EDITAS by 7.91%–12.70% accuracy improvement, indicating the generalizability of RetriGen. Overall, our study highlights the promising future of fine-tuning off-the-shelf PLMs to generate accurate assertions by incorporating external knowledge sources.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Unit Testing, Assertion Generation, Pre-trained Language Models, AI4SE

ACM Reference format:

Quanjun Zhang, Chunrong Fang, Yi Zheng, Yaxin Zhang, Yuan Zhao, Rubing Huang, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-Trained Language Models. *ACM Trans. Softw. Eng. Methodol.* 34, 7, Article 209 (August 2025), 23 pages.

<https://doi.org/10.1145/3721128>

1 Introduction

Unit testing has become a pivotal and standard practice in software development and maintenance, serving as a fundamental phase in improving software quality and reliability [11, 29]. This practice involves writing unit tests to ensure that individual components (e.g., methods, classes, and modules) perform as per their designated specifications and usage requirements. Unlike integration and system testing [6], which evaluate the system’s overall functionality, unit testing is dedicated to ensuring that individual components of the system operate as intended by the developer, enabling early detection and diagnosis of issues during software development [43]. Besides, well-designed unit tests enhance the quality of production code, minimize the costs of software failures, and facilitate debugging and maintenance processes [12, 20, 39].

Despite the significant benefits of unit testing, it is non-trivial and time-consuming to construct effective unit tests manually. For example, as shown in a prior report [9], software developers usually spend more than 15% of their time on writing unit test cases. Thus, a substantial amount of research has been dedicated to automated unit test generation, such as Randoop [35] and EvoSuite [15]. A typical unit test usually consists of two key components: (1) the test prefix involving a sequence of statements that configure the unit under test to reach a particular state, and (2) the test oracle containing an assertion that defines the expected outcome in that state [11]. Nevertheless, these test generation tools often focus on creating tests that achieve high coverage, while struggling to understand the intended program behavior and produce meaningful assertions. For instance, Almasi et al. [5] conduct an industrial evaluation of unit tests generated by EvoSuite, highlighting

that assertions in manually written tests, in contrast to automatically generated ones, are more meaningful and practical.

To tackle the assertion problem suffered by existing unit test generation tools, an increasing number of **Assertion Generation (AG)** techniques [45, 55, 59] have been proposed. Such AG techniques can be categorized into three groups: **Deep Learning (DL)**-based, retrieval-based, and integration-based ones. In particular, DL-based AG techniques (e.g., *ATLAS* [55]) handle the AG problem as a **Neural Machine Translation (NMT)** task with sequence-to-sequence learning. For example, *ATLAS* [55] takes a focal method (i.e., the method under test) and its test prefix as input to generate an assertion from scratch. For consistency with prior research [45], we refer to the input as *focal-test*, and the input-output pair as a *test-assert pair*. However, DL-based AG techniques are restricted by the quality and amount of historical test-assert pairs for training, e.g., a small corpus with only 156,760 samples for *ATLAS*. In contrast to DL-based AG techniques, given an input focal-test, retrieval-based AG techniques (e.g., IR_{ar} , RA_{adapt}^{NN} and RA_{adapt}^H [59]) retrieve the most similar focal-test from a codebase and adapt its assertion to obtain the desired outcome. However, retrieval-based AG techniques face difficulties in retrieving accurate assertions based on only lexical similarity, which is sensitive to the choice of the identifier naming of source code while ignoring the meaningful code semantics.

Recently, integration-based AG techniques (e.g., *Integration* [59] and *EDITAS* [45]) leverage both DL-based and retrieval-based approaches as basic components to retrieve or generate assertions. For example, *Integration* first utilizes IR_{ar} to retrieve similar assertions and use *ATLAS* to generate new assertions from scratch for those deemed appropriate. Meanwhile, *EDITAS* trains a neural model to generate edit actions for similar assertions retrieved by IR_{ar} . Despite their impressive performance, integration-based AG techniques still inherit the limitations of its two components, i.e., retrieving assertions by lexical similarity while overlooking code semantics and generating assertions with a small training corpus.

In this article, we propose a novel **Retrieval-Augmented AG (RetriGen)** approach equipped with a hybrid assertion retriever and a **Pre-Trained Language Model (PLM)**-based assertion generator to address limitations of prior work. Our work is inspired by the opportunity to integrate the well-known plastic surgery hypothesis [7] with recent PLMs [52] in the field of AG. To this end, we automate the plastic surgery hypothesis by fine-tuning retrieval-augmented PLMs, i.e., retrieving similar assertions from open-source projects to assist in fine-tuning PLMs for new AG. Particularly, given a focal-test, RetriGen builds a hybrid retriever to jointly search for the most relevant focal-test and its assertion from external codebases. The hybrid retriever consists of a sparse token-based retriever and a dense embedding-based retriever to consider both the lexical and semantic similarity of test-assert pairs in a unified manner. RetriGen then uses off-the-shelf PLMs as a backbone of the generator to perform the AG task by fine-tuning it with the focal-test and the retrieved external assertion as input. RetriGen is conceptually generalizable to various encoder-decoder PLMs, and we implement RetriGen on top of the recent code-aware CodeT5 model. While the retrieval-augmented generation pipeline has been explored in previous code-related studies [34, 37, 51], we are the first to investigate its effectiveness for AG by utilizing a hybrid retriever to fine-tune off-the-shelf PLMs with external knowledge sources. The distinctions between RetriGen and previous AG approaches mainly lie in both the retriever and the generator. First, prior work utilizes an assertion retriever (e.g., Jaccard similarity [45]) based on lexical matching, while RetriGen builds a hybrid retriever to search for relevant assertions jointly, demonstrating superiority over a single retriever. Besides, prior work trains an assertion generator with a basic encoder-decoder model (e.g., RNNs [55]) from a limited number of labeled training data, while RetriGen is built upon off-the-shelf language models, which are pre-trained from various open-source projects in the wild to obtain general

knowledge about programming language, thus generating optimal vector representations for unit tests.

We select six state-of-the-art AG approaches as baselines, including one DL-based approach (i.e., *ATLAS*), three retrieval-based approaches (i.e., IR_{ar} , RA_{adapt}^H , and RA_{adapt}^{NN}), and two integration-based approaches (i.e., *Integration* and its most recent follow-up *EDITAS*). We conduct extensive experiments to compare RetriGen with these baselines on two widely adopted datasets $Data_{new}$ and $Data_{old}$ in terms of both accuracy and CodeBLEU. The experimental results show that RetriGen outperforms all baselines across all datasets and metrics, with average accuracy and CodeBLEU of 57.66% and 73.24%, setting new records in the AG field. The average improvement against these baselines in accuracy and CodeBLEU score reached 50.66% and 14.14% on the two datasets. Besides, RetriGen generates 1,598 and 1,818 unique assertions that all baselines fail to predict, significantly improving the most recent approach *EDITAS* by 3.71X and 4.58X, respectively, demonstrating its superior complementarity with existing approaches. Furthermore, we further explore the influence of each component and observe that all components positively contribute to the performance of RetriGen, e.g., an accuracy improvement of 7.73% brought by the hybrid retriever. Finally, we implement RetriGen with CodeBERT, GraphCodeBERT, UniXcoder, and CodeGPT and find these variants achieve an accuracy of 57.69%–60.25% and 46.96%–48.58% on $Data_{old}$ and $Data_{new}$, outperforming the most recent approach *EDITAS* by 7.91%–12.70% and 5.86%–9.51%, respectively. The results show that RetriGen is universal and is effectively adapted to different PLMs with sustaining advancements, highlighting the applicability and generalizability of RetriGen in practice.

To sum up, the contributions of this article are as follows:

- We introduce a generic yet effective retrieval-augmented AG paradigm in the unit testing scenario with a hybrid assertion retriever and a PLM-based assertion generator. The framework is generic and can be integrated with different encoder–decoder PLMs.
- We propose RetriGen, which utilizes a sparse token-based retriever and a dense embedding-based retriever to jointly search for the relevant assertion based on both lexical and semantic matching. We adopt a code-aware PLM CodeT5 as the foundation model for the assertion generator. To the best of our knowledge, RetriGen is the first attempt to fine-tune recent PLMs with the advance of external codebases for the crucial AG task.
- We extensively evaluate RetriGen against six prior AG approaches on two datasets and two metrics. The experimental results demonstrate that RetriGen significantly outperforms all baselines, with an average accuracy improvement of 58.81% and 42.51% on $Data_{old}$ and $Data_{new}$.
- We release the relevant materials in our experiment to facilitate follow-up AG studies at the repository,¹ including datasets, scripts, models, and generated assertions.

2 Background and Related Work

2.1 Deep AG

With the success of DL, researchers have increasingly been utilizing advanced DL techniques to automate a variety of software engineering tasks [54, 58, 62]. For example, Watson et al. [55] propose *ATLAS*, the first DL-based AG approach that utilizes deep neural networks to learn correct assertions from existing test–assert pairs. Yu et al. [59] propose two retrieval-based approaches to generate assertions by searching for similar assertions given a focal-test, namely **Information Retrieval (IR)**-based assertion retrieval (IR_{ar}) and retrieved-assertion adaptation (RA_{adapt}).

¹<https://github.com/iSEngLab/RetriGen>

IR_{ar} retrieves the assertion with the highest similarity to the given focal-test using measures like Jaccard similarity. As IR_{ar} may not always retrieve completely accurate assertions, RA_{adapt} attempts to revise the retrieved assertions by replacing tokens within them. Two strategies are proposed for determining replacement values, i.e., a heuristic-based approach RA_{adapt}^H and a neural network-based approach RA_{adapt}^{NN} . Furthermore, Yu et al. [59] propose an integration-based approach called *Integration* by integrating IR and DL techniques. Building on *Integration*, Sun et al. [45] introduce EDITAS by searching for a similar focal-test, which is then modified by a neural model. Unlike prior AG studies retrieving relevant assertions [59] with token matching or training generator from scratch [55] with historical test–assert pairs, our work attempts to utilize PLMs as an assertion generator with the help of a hybrid assertion retriever for more effective retrieval and generation.

The literature has also witnessed several studies [32, 33, 48], exploring the use of PLMs like T5 [40] in the field of AG. For instance, Mastropaolo et al. [32] pre-train a T5 model and fine-tune it on four code-related tasks: bug-fixing, mutant generation, AG, and code summarization. These studies typically pre-train language models from scratch and fine-tune them on multiple downstream tasks. In contrast, our work attempts to propose a specific AG approach by leveraging off-the-shelf PLMs. Nashid et al. [34] propose CEDAR, a **Large Language Model (LLM)**-based approach to apply retrieval-based demonstration selection strategy for program repair and AG. RetriGen and CEDAR are fundamentally distinct regarding their learning paradigms, retrievers, and generators. First, CEDAR utilizes few-shot learning with prompt engineering, while RetriGen leverages fine-tuning with augmented inputs. Second, CEDAR utilizes a single retriever, while RetriGen builds a hybrid retriever to identify relevant assertions jointly, indicating superiority over a single retriever. Third, CEDAR queries APIs from a black-box LLM Codex with 12 billion parameters, whereas RetriGen fine-tunes an open-source PLM CodeT5 with only 220 million parameters. Recently, Zhang et al. [67] conduct an empirical study to explore the potential of several PLMs for generating assertions in a fine-tuning scenario.

2.2 PLM

Recently, researchers have explored the capabilities of PLMs to revolutionize various code-related tasks [13, 64], such as code review [28, 49], test generation [8, 17, 25, 42, 46, 53, 57, 61], and program repair [56, 63, 66].

There exist three main categories based on model architectures. (1) Encoder-only PLMs, e.g., CodeBERT [14], train the encoder part of the Transformer with masked language modeling, thus suitable for code understanding. (2) Decoder-only PLMs, e.g., CodeGPT [31], train the decoder part of the Transformer with unidirectional language modeling, thus suitable for auto-regressive generation. (3) Encoder–decoder PLMs, e.g., CodeT5 [52], train both encoder and decoder parts of the Transformer with denoising objectives, thus suitable for sequence-to-sequence generation.

In our work, the foundation model selection of RetriGen is limited to encoder–decoder PLMs, as assertions are generated in a sequence-to-sequence learning manner. Following prior work [16, 51, 65, 69], we consider CodeT5, a generic code-aware language model that is pre-trained on a large code corpus, achieving state-of-the-art performance in both code understanding and generation tasks. CodeT5 is the most popular encoder–decoder PLM that is adopted by previous fine-tuning-based sequence-to-sequence code generation tasks [50, 60, 70]. Besides, CodeT5 is trained with CodeSearchNet [31] without test code snippets, thus avoiding the data leakage issue.

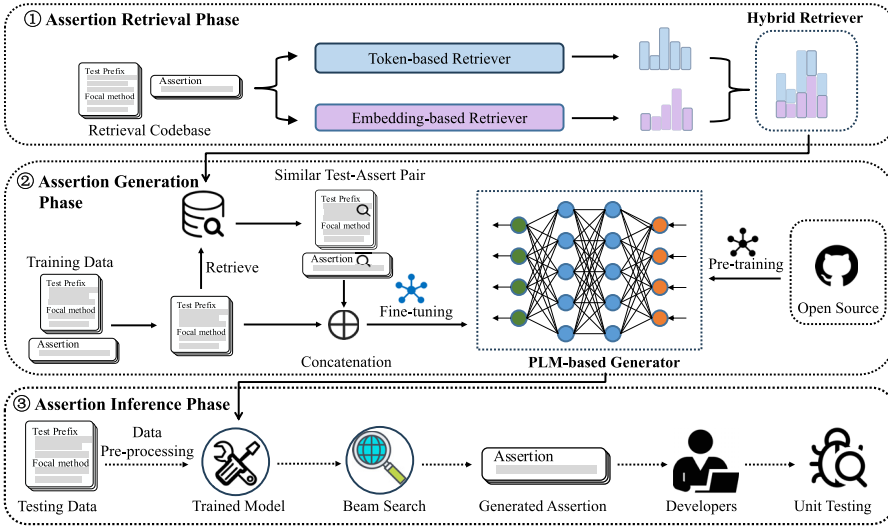


Fig. 1. Framework overview of RetriGen.

2.3 IR for SE

IR refers to the process of identifying relevant information from a large collection of data. IR has been extensively applied to a range of code-related tasks, such as fault localization [10] and test case prioritization [38]. Such approaches search for the object that best matches the given query from the database based on different similarity coefficients, such as Jaccard similarity. Besides, the literature has seen an increasing number of studies performing generation tasks with the retrieval-augmented paradigm, such as code repair [34, 51] and code summarization [27, 37]. This paradigm improves the quality of generated results by grounding the model with external knowledge sources to supplement the PLM's internal representation of information [26]. In our work, inspired by the intuition of retrieval-augmented generation in the PLM domain, we focus on the AG problem and propose RetriGen to fine-tune a PLM-based assertion generator with a hybrid assertion retriever.

3 Approach

The framework overview of RetriGen is illustrated in Figure 1, which consists of three phases. In the assertion retrieval phase, RetriGen identifies a similar assertion from the external codebase based on lexical and semantical similarity calculated by a token-based retriever and an embedding-based retriever. In the assertion generator training phase, RetriGen is first pre-trained with millions of code snippets from open-source projects and then fine-tuned with the retrieval-augmented labeled pairs, i.e., the focal-test and the retrieved assertion as input and the correct assertion as output. In the assertion inference phase, after the generator is well trained, RetriGen leverages the beam search strategy to generate a ranked list of candidate assertions, and return the one with the highest probability of being correct.

3.1 Problem Definition

Similar to the DL-based approach *ATLAS*, RetriGen regards AG as an NMT task based on the encoder-decoder Transformer architecture. Suppose $\mathcal{D} = (FT_i, A_i)_{i=1}^{|\mathcal{D}|}$ be a unit testing dataset consisting of $|\mathcal{D}|$ test-assert pairs, where FT_i and A_i are the i th focal-test and the corresponding

assertion, respectively. The sequence-to-sequence assertion generator attempts to predict A_i from FT_i in an auto-regressive manner, which is formally defined as follows:

Definition 1 (Deep AG). Given a focal-test input $FT_i = [ft_1, \dots, ft_m]$ with m code tokens and an assertion output $A_i = [a_1, \dots, a_n]$ with n code tokens, the problem of AG is defined as maximizing the conditional probability, i.e., the likelihood of A_i being the correct assertion:

$$P_{\theta}(A_i|FT_i) = \prod_{j=1}^n P_{\theta}(a_j|a_1, \dots, a_{j-1}; ft_1, \dots, ft_m).$$

However, different from ATLAS generating new assertions from scratch, RetriGen takes the focal-test and an additional retrieved assertion as input and the correct assertion as output. Thus, on top of Definition 1, suppose $C = (FT'_j, A'_j)_{j=1}^{|C|}$ be an external codebase containing a large collection of historical test–assert pairs, where FT'_j and A'_j denote the j th previous focal-test and the corresponding assertion. Then, with the codebase C , the RetriGen can be defined as follows:

Definition 2 (RetriGen). Given a focal-test FT_i in $\mathcal{D}$, the retriever searches for the most relevant focal-test FT'_j from the codebase C , and its assertion $A'_j = [a'_1, \dots, a'_z]$ with z code tokens. Then the original focal-test input FT_i is augmented with the retrieved assertion to form a new input sequence $\hat{FT}_i = FT_i \oplus A'_j$, where $\oplus$ denotes the concatenation operation. Finally, the assertion generator attempts to generate A_i from $\hat{FT}_i$ by learning the following the probability:

$$P_{\theta}(A_i|\hat{FT}_i) = \prod_{j=1}^n P_{\theta}(a_j|a_1, \dots, a_{j-1}; \underbrace{FT_i}_{\text{Original}}; \underbrace{a'_1, \dots, a'_z}_{\text{Augmented}}).$$

3.2 Hybrid Assertion Retrieval Component

RetriGen utilizes a hybrid strategy that integrates a token-based retriever and an embedding-based retriever, allowing it to account for both the lexical and semantic similarities of test–assert pairs.

3.2.1 Token-Based Retriever. RetriGen utilizes the sparse strategy IR as the token-based retriever to identify an assertion that closely resembles the query focal-test through lexical matching, which has been proven to be effective in previous AG studies [45, 59]. In particular, RetriGen tokenizes all focal-tests in both the dataset $\mathcal{D}$ and the codebase C and eliminates duplicate tokens to enhance the efficiency of the retrieval process. Given a query focal-test FT_i in $\mathcal{D}$, RetriGen computes the lexical similarity between FT_i and all focal-tests in C using the Jaccard coefficient as the similarity measure. The Jaccard coefficient is a commonly employed metric for assessing the similarity between two sparse vector representations by considering their overlapping and unique tokens. The calculation of Jaccard similarity is defined in Formula (1), where $S(FT_i)$ and $S(FT_j)$ represent the sets of code tokens for the two focal-tests FT_i and FT_j , respectively.

$$Jac(FT_i, FT_j) = \frac{|S(FT_i) \cap S(FT_j)|}{|S(FT_i) \cup S(FT_j)|}. \quad (1)$$

3.2.2 Embedding-Based Retriever. RetriGen utilizes the pre-trained CodeLlama [41] as the embedding-based retriever, to identify the most similar assertion based on semantic similarity. CodeLlama is an advanced language model pre-trained with common programming languages that support code understanding and generation tasks. Unlike previous studies training the dense

retriever from scratch [24], we directly employ the checkpoint of CodeLlama from Hugging Face without any fine-tuning in our work as it is already trained with a mass of testing code to get meaningful vector embeddings for the query focal-test in the unit testing scenario.

In particular, RetriGen first splits the source code of the focal-test into a sequence of code tokens and utilizes CodeLlama to convert these tokenized elements into vector representations. RetriGen then computes the Cosine similarity between the embeddings of two focal-tests to assess their semantic relevance. Cosine similarity has been widely employed in prior research for measuring the semantic relationship between two dense vectors [36]. It is calculated by taking the cosine of the angle between two vectors, which is the dot product of the vectors divided by the product of their magnitudes. Formula (2) presents the Cosine similarity calculation, where $\mathbf{FT}_i$ and $\mathbf{FT}_j$ represent the embeddings of focal-tests FT_i and FT_j .

$$\text{Cos}(FT_i, FT_j) = \frac{\mathbf{FT}_i \cdot \mathbf{FT}_j}{\|\mathbf{FT}_i\| \|\mathbf{FT}_j\|}. \quad (2)$$

3.2.3 Hybrid Retriever. Our sparse token-based retriever primarily relies on identifier naming in the source code, making it more sensitive to lexical choices rather than the underlying code semantics. In contrast, our dense embedding-based retriever focuses more on the semantic representation of the code rather than its literal syntax. Thus, to incorporate both lexical and semantic information, we adopt a hybrid approach that integrates the two retrievers in a unified manner. The combined similarity score between two focal-tests is computed as shown in Formula (3), where λ serves as a weighting factor to balance the contributions of the two retrievers, with its default value set to 1. Based on this combined similarity score, we select the top-1 relevant test-assert pair (FT'_i, A'_i) to guide the assertion generator for generating assertions.

$$\text{Sim}(FT_i, FT_j) = \text{Jac}(FT_i, FT_j) + \lambda \text{Cos}(FT_i, FT_j). \quad (3)$$

3.3 AG Component

RetriGen utilizes CodeT5 as the foundation model to generate the assertion A_i , leveraging both the focal-test FT_i and the externally retrieved assertion A'_j as input. The AG component is general to a mass of existing PLMs, and we select CodeT5 as it is a code-aware encoder-decoder PLM optimized for source code and has demonstrated effectiveness in various sequence-to-sequence code generation tasks, such as program repair [16, 65].

Input Representation. As discussed in Section 3.1, the retrieval-augmented focal-test input to RetriGen is represented as $\hat{FT}_i = FT_i \oplus A'_j$ with the concatenation operation $\oplus$. Thus, the generator is trained to learn the transformation patterns from the retrieval-augmented input $\hat{FT}_i$ to the output A_i by the sequence-to-sequence learning.

Model Architecture. The generation model of RetriGen is composed of an encoder stack and a decoder stack, culminating in a linear layer with softmax activation. First, RetriGen splits the source code of the input $\hat{FT}_i$ into subwords using a code-specific **Byte-Pair Encoding (BPE)** tokenizer, which has been pre-trained on eight widely used programming languages and is specifically designed for tokenizing source code [52]. The BPE tokenizer is able to break down rare tokens into meaningful subword units based on their frequency distribution to address the common out-of-vocabulary problem in code-related tasks. Second, RetriGen employs a word embedding stack to generate representation vectors for tokenized focal-test functions, allowing it to capture the semantic meaning of the code tokens as well as their positional relationships within the code snippet. Third, RetriGen inputs the vectors into an encoder stack to obtain the hidden state, which is subsequently passed to a decoder stack for further processing. Finally, the output from the decoder

stack is passed through a linear layer with softmax activation, generating a probability distribution over the vocabulary.

Loss Function. The AG model receives $\hat{F}\hat{T}_i$ as input and produces the correct assertion A_i by learning the mapping between $\hat{F}\hat{T}_i$ and A_i . Thus, the model's parameters are adjusted through the training dataset, with the goal of improving the mapping by maximizing the conditional probability $P_\theta(A_i|\hat{F}\hat{T}_i)$. RetriGen employs the common cross-entropy as the loss function to train the assertion generator $\mathcal{L}$, which is widely adopted in previous code-related studies [14, 52]. The cross-entropy loss is minimized between each position in the predicted assertion and each position in the ground-truth assertion, defined as follows.

$$\mathcal{L} = - \sum_{i=1}^{|\mathcal{D}|} \log(P_\theta(A_i|\hat{F}\hat{T}_i)). \quad (4)$$

3.4 Assertion Inference

During the model inference phase, once the generation model is effectively trained, given a focal-test as the input, RetriGen employs the beam search strategy to produce a ranked list of assertion candidates based on the vocabulary's probability distribution. In particular, beam search [55] is a widely used strategy for identifying the highest-scoring candidate assertions by progressively prioritizing the top- k probable tokens according to their predicted likelihood scores. The correctness of the generated candidate assertion can be automatically evaluated by comparing it with ground-truth assertions or manually inspected by test experts for deployment in the unit testing pipeline.

4 Experimental Setup

4.1 Research Questions (RQs)

To evaluate the performance of RetriGen, we address the following three RQs:

- RQ1: How does RetriGen compare to state-of-the-art AG approaches?
- RQ2: To what extent do different choices influence the overall effectiveness of RetriGen?
- RQ3: What is the generalizability of RetriGen when utilizing other advanced PLMs?

4.2 Datasets

Following EDITAS [45], we utilize two popular large-scale datasets, i.e., $Data_{old}$ and $Data_{new}$, to evaluate RetriGen and the baselines. These datasets have been widely used in previous deep AG studies [21, 45, 55, 59], including all of our baselines. A brief introduction to both datasets is provided below.

- (1) $Data_{old}$. $Data_{old}$ is initially constructed by Watson et al. [55] to propose *ATLAS* [55], the first deep AG approach. Watson et al. [55] collect a dataset of 2.5 million test methods within GitHub, each comprising test prefixes and their corresponding assertion statements. In $Data_{old}$, every test method is linked to a specific focal method, which represents the production code being tested. Watson et al. then perform pre-processing to remove test methods with token lengths exceeding 1K and to filter out assertions that contain *unknown* tokens not present in the focal-test or the vocabulary.
- (2) $Data_{new}$. However, $Data_{old}$ excludes assertions containing unknown tokens to oversimplify the AG problem, thus being unsuitable to reflect the real-world data distribution. Thus, Yu et al. [59] address this issue by creating an extended dataset, referred to as $Data_{new}$, which includes additional 108,660 samples that are excluded due to unknown tokens in $Data_{old}$.

Table 1. Detailed Statistics of Each Type in $Data_{new}$ and $Data_{old}$

AssertType	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
$Data_{old}$	15,676	7,866 (50%)	2,783 (18%)	1,441 (9%)	1,162 (7%)	1,006 (6%)	798 (5%)	307 (2%)	311 (2%)	2 (0%)
$Data_{new}$	26,542	12,557 (47%)	3,652 (14%)	3,532 (13%)	1,284 (5%)	1,071 (4%)	735 (3%)	362 (1%)	319 (1%)	3,030 (11%)

As a result, $Data_{old}$ and $Data_{new}$ contain a total of 156,760 and 265,420 samples, respectively. These datasets are divided into training, validation, and test sets using an 8:1:1 ratio, as done by Watson et al. [55] and Yu et al. [59]. In this article, we strictly adhere to the replication package provided by prior work [45, 55, 59]. Table 1 presents the statistics of the testing sets for both datasets and their distribution across various assertion types.

4.3 Baselines

To address the above-mentioned RQs, we compare RetriGen against six state-of-the-art AG approaches, including DL-based, retrieval-based, and integration-based ones. First, we include *ATLAS*, the first and classical AG technique that utilizes a sequence-to-sequence model to generate assertions from scratch. *ATLAS* is the most relevant baseline as both *ATLAS* and RetriGen consider AG as an NMT task based on the encoder-decoder Transformer architecture. Second, we incorporate three existing retrieval-based AG techniques: IR_{ar} , RA_{adapt}^H , and RA_{adapt}^{NN} . Finally, we consider one state-of-the-art integration-based approach *Integration*, and its most recent follow-up *EDITAS*, as detailed in Section 2.1. It is worth noting that we exclude CEDAR as a baseline mainly due to the severe data leakage issue of the utilized black-box LLM Codex. LLMs are proprietary, and their training details (e.g., pre-training datasets) are not publicly disclosed, making it difficult to ensure whether models have been exposed to the evaluation dataset during training [44, 47, 68].

4.4 Evaluation Metrics

Following prior AG work [45, 55, 59], we consider two metrics to evaluate the performance of RetriGen and baselines, which are widely adopted in code-related studies [49, 51, 65].

Accuracy. Accuracy is defined as the proportion of focal-tests for which assertions generated by AG approaches match the ground-truth ones and is utilized by all baseline methods [45, 55, 59]. An assertion is deemed correct only when it precisely aligns with the ground truth at the token level.

CodeBLEU. Apart from accuracy, BLEU is utilized by previous AG studies [55, 59] to evaluate how closely the generated assertion matches the ground truth. However, BLEU is initially developed for natural language through token-level matching, overlooking significant lexical and semantic aspects of source code. In this work, we consider a code-aware variant of BLEU, i.e., CodeBLEU, which is designed particularly for code generation tasks [31]. Compared with BLEU, CodeBLEU is more suitable for the AG task as it further incorporates lexical similarity with the AST information and semantic similarity with a dataflow structure.

4.5 Implementation Details

We implement our experiments, including RetriGen, using the PyTorch framework [4]. We employ the Hugging Face implementation [1] of the studied PLMs and all hyper-parameters are taken from default values. We utilize the training set as the search codebase, consistent with prior deep AG studies [45, 59]. To prevent data leakage, we perform queries on the datasets to verify that there is no overlap between the evaluation and training sets. This strategy ensures that the retriever and

Table 2. Comparisons of RetriGen with State-of-the-Art AG Approaches

Approach	<i>Data_{old}</i>		<i>Data_{new}</i>	
	Accuracy	CodeBLEU	Accuracy	CodeBLEU
<i>ATLAS</i>	31.42% ($\uparrow 103.98\%$)	63.60% ($\uparrow 25.46\%$)	21.66% ($\uparrow 136.47\%$)	37.91% ($\uparrow 75.89\%$)
<i>IR_{ar}</i>	36.26% ($\uparrow 76.75\%$)	71.03% ($\uparrow 12.33\%$)	37.90% ($\uparrow 35.15\%$)	62.67% ($\uparrow 6.40\%$)
<i>RA_{adapt}^H</i>	40.97% ($\uparrow 56.43\%$)	72.46% ($\uparrow 10.12\%$)	39.65% ($\uparrow 29.18\%$)	63.66% ($\uparrow 4.74\%$)
<i>RA_{adapt}^{NN}</i>	43.63% ($\uparrow 58.13\%$)	72.12% ($\uparrow 10.64\%$)	40.53% ($\uparrow 17.40\%$)	63.19% ($\uparrow 5.52\%$)
<i>Integration</i>	46.54% ($\uparrow 37.71\%$)	73.29% ($\uparrow 8.87\%$)	42.20% ($\uparrow 21.37\%$)	63.00% ($\uparrow 5.84\%$)
<i>EDITAS</i>	53.46% ($\uparrow 19.88\%$)	77.00% ($\uparrow 3.62\%$)	44.36% ($\uparrow 15.46\%$)	64.40% ($\uparrow 3.54\%$)
RetriGen	64.09%	79.79%	51.22%	66.68%

$\uparrow$ denotes performance improvement of RetriGen against state-of-the-art baselines. Bold values indicate the best performance in each category.

generator are not privy to ground-truth assertions. During training, we set the batch size to 8, the maximum lengths of input to 512, the maximum lengths of output to 256, and the learning rate to $2e-5$. All training and evaluations are performed on one Ubuntu 20.04 server with two NVIDIA GeForce RTX 4090 GPUs.

5 Evaluation and Results

5.1 RQ1: Comparison with State-of-the-Arts

Experimental Design. In RQ1, we aim to evaluate the effectiveness of assertions generated by RetriGen. We compare RetriGen with six state-of-the-art AG techniques, including *ATLAS*, *IR_{ar}*, *RA_{adapt}^H*, *RA_{adapt}^{NN}*, *Integration*, and *EDITAS*. To ensure a fair comparison, we employ the same training set to train baselines and RetriGen, and we provide the same evaluation set to all approaches for evaluation. We calculate the accuracy and CodeBLEU of all baselines and RetriGen by comparing generated assertions and human-written assertions. According to prior work [45, 59], we directly utilize reported results from their original paper rather than re-executing baselines, thereby minimizing potential risks.

Results and Analysis. Table 2 presents the comparison results of RetriGen and six baselines in terms of the prediction accuracy and CodeBLEU score. Overall, RetriGen achieves a remarkable performance with a prediction accuracy of 57.66% and a CodeBLEU score of 73.24% on average for two datasets, with an improvement of 50.66% and 14.14% against all previous baselines, respectively.

In particular, when compared with the DL-based approach *ATLAS*, RetriGen achieves a prediction accuracy of 64.09% and 51.22% on the *Data_{old}* and *Data_{new}* datasets, yielding a remarkable improvement of 103.98% and 136.47%, respectively. Similarly, RetriGen exhibits substantial gains in the CodeBLEU metric, achieving improvements of 25.46% and 75.89% on average. It is worth noting that *ATLAS* is the most relevant to our RetriGen, as both approaches address AG as an NMT task based on the encoder-decoder Transformer architecture. The significant improvement against *ATLAS* indicates the advantage of our PLM-based assertion generator, which is pre-trained with millions of code snippets, thus getting rid of the limited number of training samples. When compared with retrieval-based approaches, RetriGen attains an average accuracy improvement of 55.95%, 42.81%, and 37.76% against *IR_{ar}*, *RA_{adapt}^H*, and *RA_{adapt}^{NN}* across the two datasets. Similarly, in terms of the CodeBLEU metric, the improvement still reaches 9.37%, 7.43%, and 8.08%. When compared with the two advanced integration-based approaches *Integration* and *EDITAS*, RetriGen

Table 3. Detailed Statistics of RetriGen and Baselines for Each Assert Type

Dataset	Approach	Total	AssertType								
			Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
Data _{old}	ATLAS	4,925 (31%)	2,501 (32%)	966 (35%)	248 (17%)	598 (51%)	229 (23%)	236 (30%)	100 (33%)	47 (15%)	0 (0%)
	IR _{ar}	5,684 (36%)	2,957 (38%)	1,039 (37%)	449 (31%)	439 (38%)	314 (31%)	285 (36%)	111 (36%)	89 (29%)	1 (50%)
	RA ^H _{adapt}	6,423 (41%)	3,300 (42%)	1,151 (41%)	536 (37%)	553 (48%)	335 (33%)	316 (40%)	120 (39%)	111 (36%)	1 (50%)
	RA ^{NN} _{adapt}	6,839 (44%)	3,509 (45%)	1,225 (44%)	551 (38%)	610 (52%)	342 (34%)	341 (43%)	134 (44%)	126 (41%)	1 (50%)
	Integration	7,295 (47%)	3,714 (47%)	1,333 (48%)	546 (38%)	724 (62%)	348 (35%)	352 (44%)	148 (48%)	129 (41%)	1 (50%)
	EdrtAS	8,380 (53%)	4,131 (53%)	1,581 (57%)	526 (36%)	807 (69%)	577 (57%)	469 (59%)	167 (54%)	122 (39%)	0 (0%)
	RetriGen	10,042 (64%)	5,027 (64%)	1,791 (64%)	804 (56%)	820 (71%)	654 (65%)	568 (71%)	184 (60%)	194 (62%)	0 (0%)
	Data _{new}	ATLAS	5,749 (22%)	2,900 (23%)	619 (17%)	537 (15%)	388 (30%)	126 (12%)	85 (12%)	47 (13%)	37 (12%)
IR _{ar}		10,059 (38%)	4,664 (37%)	1,436 (39%)	1,070 (30%)	600 (47%)	394 (37%)	286 (39%)	147 (41%)	113 (35%)	1,349 (45%)
RA ^H _{adapt}		10,525 (40%)	4,882 (39%)	1,487 (41%)	1,142 (32%)	651 (51%)	403 (38%)	297 (40%)	154 (43%)	121 (38%)	1,388 (46%)
RA ^{NN} _{adapt}		10,758 (41%)	4,988 (40%)	1,526 (42%)	1,161 (33%)	691 (54%)	401 (37%)	308 (42%)	162 (45%)	126 (39%)	1,395 (46%)
Integration		11,201 (42%)	5,248 (42%)	1,566 (43%)	1,196 (34%)	711 (55%)	401 (37%)	313 (43%)	162 (45%)	128 (40%)	1,476 (49%)
EdrtAS		11,773 (44%)	5,339 (42%)	1,702 (47%)	1,304 (37%)	800 (62%)	523 (49%)	376 (51%)	172 (47%)	139 (44%)	1,418 (47%)
RetriGen		13,590 (51%)	6,294 (50%)	1,860 (51%)	1,588 (45%)	840 (65%)	609 (57%)	433 (59%)	195 (54%)	176 (55%)	1595 (53%)

Bold values indicate the best performance in each category.

improves the prediction accuracy by 37.71% and 19.88% on *Data_{old}*, and 21.37% and 15.46% on *Data_{new}*, respectively.

Effectiveness on Different Assertion Types. Table 3 presents the performance of RetriGen and baselines on different types of assertions. The rows denote seven approaches (i.e., six baselines and RetriGen) and two datasets (i.e., *Data_{old}* and *Data_{new}*). The columns denote nine different assertion types, with each cell displaying the number of correctly generated assertions and their corresponding ratios in parentheses. From Table 3, RetriGen outperforms all baselines on all standard JUnit assertion types across both datasets. For example, for the most common *Equals* type, RetriGen correctly generates 5,027 and 6,294 assertions on two datasets, with a prediction accuracy rate of 64% and 50%, improving the best-performing one *EDITAS* by 20.75% and 19.05%, respectively. Besides, in the case of *Other* assertion type, RetriGen outperforms existing baselines, except in the *Data_{old}* dataset, which has only two samples and is too small to yield convincing results. For example, RetriGen generates 1,595 correct assertions on *Data_{new}* with an improvement of 8.16% against the best-performing one *Integration*, indicating the capability of RetriGen in addressing such non-standard JUnit assertion types. In summary, the experimental results demonstrate the generality of RetriGen in generating various types of assertions, including both standard and non-standard JUnit types.

Overlap Analysis. To explore the extent to which RetriGen complements prior studies, Figure 2 presents the number of overlapping assertions generated by different AG techniques. Due to page limit, we select one DL-based technique *ATLAS*, one retrieval-based technique *IR_{ar}*, and one integration-based technique *Integration*, as well as the most recent *EDITAS* for comparison. As shown in Figure 2, RetriGen generates 1,598 unique assertions that other AG approaches fail to generate on *Data_{old}*, which are 1,473, 1,573, 1,535, 1,259 more than *ATLAS*, *IR_{ar}*, *Integration*, and *EDITAS*, respectively. The improvement achieves substantial multiples of 11.78X, 62.92X, 24.37X, and 3.71X over these respective baselines. Similarly, on the *Data_{new}* dataset, RetriGen showcases its effectiveness by generating 1,818 unique assertions, which is 1,528 more than the baselines on average, with a remarkable 30.38X improvement. Overall, the results demonstrate the superior capability of RetriGen in generating unique assertions, indicating the potential to complement existing AG techniques.

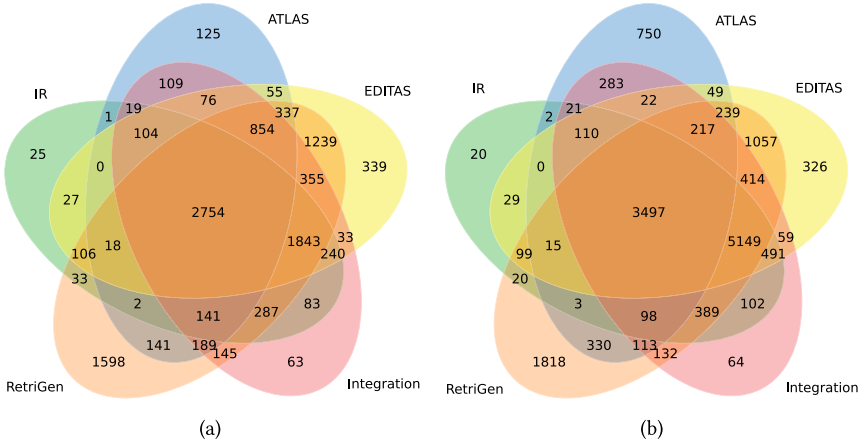


Fig. 2. Overlaps of assertions generated by different AG approaches: (a) $Data_{old}$ and (b) $Data_{new}$.

Answer to RQ1. Our comparison results demonstrate that: (1) RetriGen significantly outperforms all baselines in terms of accuracy and CodeBLEU across both datasets, e.g., with an average accuracy improvement of 58.81% on $Data_{old}$ and 42.51% on $Data_{new}$; (2) RetriGen consistently outperforms all baselines on all standard JUnit assertion types across both datasets, e.g., improving EDITAS by 20.75% and 19.05% in generating assertions for the *Equals* type; (3) RetriGen generates a large number of unique assertions that all baselines fail to generate across both datasets, e.g., 1,598 and 1,818 unique ones on $Data_{old}$ and $Data_{new}$, with an improvement of 3.71X and 4.58X against the most recent baseline ATLAS.

5.2 RQ2: Impact Analysis

Experimental Design. In RQ2, we attempt to investigate the impacts of different components in RetriGen. RetriGen employs a hybrid retriever to search for a relevant test–assert pair, which consists of a token-based retriever and an embedding-based retriever. To illustrate the importance of each component, we compare RetriGen with three of its variants: (1) RetriGen_{none} that generates assertions without any retriever, which is similar to ATLAS; (2) RetriGen_{token} that generates assertions with only a token-retriever; (3) RetriGen_{embed} that generates assertions with only an embedding-retriever.

Results and Analysis. Table 4 displays the comparison results of RetriGen under different components. First, we find that RetriGen outperforms all variants in terms of accuracy and CodeBLEU across both datasets, indicating the benefits of each component. For example, on the $Data_{old}$ dataset, the hybrid retriever of RetriGen improves the prediction accuracy by 9.16%, 1.14%, and 1.55%, against RetriGen_{none}, RetriGen_{token}, and RetriGen_{embed}, respectively. Similarly, the average improvement reaches 2.46%, 3.03%, and 2.09% under the remaining three settings, i.e., accuracy on $Data_{old}$, CodeBLEU on $Data_{old}$, and $Data_{new}$. Second, we find that without any retriever, RetriGen still achieves remarkable performance, with a prediction accuracy of 58.71% and 48.19% on two datasets. Compared with the most relevant baseline ATLAS, the improvement reaches 86.86% and 122.48%, highlighting RetriGen’s advanced capability in overcoming the challenges posed by limited training datasets. It is worth noting that although the improvement of RetriGen over RetriGen_{token} and RetriGen_{embed} is not as significant as its improvement over RetriGen_{none}, given that three

Table 4. Effectiveness of RetriGen with Different Retriever Choices

Approach	<i>Data_{old}</i>		<i>Data_{new}</i>	
	Accuracy	CodeBLEU	Accuracy	CodeBLEU
RetriGen _{none}	58.71% (↑9.16%)	74.91% (↑6.51%)	48.19% (↑6.29%)	63.40% (↑5.17%)
RetriGen _{token}	63.37% (↑1.14%)	78.54% (↑1.59%)	51.15% (↑0.14%)	66.54% (↑0.21%)
RetriGen _{embed}	63.11% (↑1.55%)	79.01% (↑0.99%)	50.74% (↑0.95%)	66.09% (↑0.89%)
RetriGen	64.09%	79.79%	51.22%	66.68%

↑denotes performance improvement of the hybrid retriever against baselines. Bold values indicate the best performance in each category.

variants have achieved state-of-the-art results, the improvement brought by our hybrid retriever is valuable in practice, leading to a new higher baseline of deep AG performance.

Answer to RQ2. Our impact analysis indicates that all components (such as token-based and embedding-based retrievers) positively contribute to the effectiveness of RetriGen across two metrics, leading to new records on two widely adopted datasets.

5.3 RQ3: Generalizability of RetriGen

Experimental Design. RQ1 and RQ2 have proven that RetriGen outperforms the baselines when utilizing CodeT5 as the foundational model. To further explore the impact of various PLMs on the performance of RetriGen, we utilize four additional advanced PLMs for the AG task: CodeBERT, GraphCodeBERT, UniXcoder, and CodeGPT. Particularly, CodeBERT [14] is a bi-modal PLM for both programming language and natural language understanding, pre-trained with masked language modeling and replaced token detection. GraphCodeBERT [19] is a successor of CodeBERT by incorporating two structure-aware pre-training tasks, i.e., edge prediction and node alignment. UniXcoder [18] is a unified cross-modal PLM designed to enhance both code understanding and generation capabilities with two pre-training tasks, i.e., multi-modal contrastive learning, and cross-modal generation. All these PLMs are pre-trained with source code, publicly accessible, and medium-scale, thus suitable for fine-tuning in the AG task.

To implement RetriGen, for encoder-decoder PLMs, like UniXcoder, we use Test-Assertion Pairs to train both encoder and decoder components, thus learning hidden mappings between two sequences. For encoder-only PLMs, like CodeBERT and GraphCodeBERT, we initialize a new decoder from scratch to construct an encoder-decoder architecture for fine-tuning. For decoder-only PLMs, like CodeGPT, we directly leverage GPT's capabilities to generate subsequent assertions from previous focal-methods in an auto-regressive manner.

Results and Analysis. Table 5 presents the comparison performance of RetriGen with different PLMs as assertion generators. Overall, RetriGen consistently achieves impressive performance across all PLMs and two datasets with an average accuracy of 54.29% and CodeBLEU score of 70.88%. Particularly, when comparing the performance among different PLMs, we find the default generator of RetriGen (i.e., CodeT5) achieves better performance than the other four PLMs for all metrics and datasets. For example, on the *Data_{old}* dataset, the prediction accuracy improvement against CodeBERT, GraphCodeBERT, UniXcoder, and CodeGPT reaches 6.37%, 8.32%, 8.92%, and 11.09%, respectively. Similarly, CodeT5 outperforms other PLMs by an average of 6.95%, 3.62%, and 4.90% for other three settings, i.e., accuracy on *Data_{old}*, accuracy, and CodeBLEU on *Data_{new}*. The

Table 5. Effectiveness of RetriGen with Different PLMs as Assertion Generators

Approach	<i>Data_{old}</i>		<i>Data_{new}</i>	
	Accuracy	CodeBLEU	Accuracy	CodeBLEU
Graphco Debert	60.25% (↑6.37%)	77.42% (↑3.06%)	46.96% (↑9.07%)	63.05% (↑5.76%)
Unixcoder	59.17% (↑8.32%)	75.77% (↑5.31%)	48.58% (↑5.43%)	63.67% (↑4.73%)
CodeBERT	58.84% (↑8.92%)	77.14% (↑3.44%)	47.92% (↑6.89%)	63.46% (↑5.07%)
CodeGPT	57.69% (↑11.09%)	77.70% (↑2.69%)	48.13% (↑6.42%)	64.08% (↑4.06%)
CodeT5 (RetriGen)	64.09%	79.79%	51.22%	66.68%

↑denotes performance improvement of CodeT5 against baselines. Bold values indicate the best performance in each category.

possible reason for the advance of CodeT5 lies in the model architecture. Similar to *ATLAS*, CodeT5 features an encoder–decoder Transformer architecture, which has been shown to effectively support code generation tasks in prior work [52]. However, encoder-only PLMs (such as GraphCodeBERT) necessitate a decoder for generation tasks, where the decoder starts from scratch and does not leverage the pre-training dataset. Thus, it is natural and effective to employ CodeT5 as the backbone of RetriGen.

When comparing different PLMs against previous AG approaches (shown in Table 2), we find that all PLMs are able to achieve impressive performance. For example, five investigated PLMs achieve 57.69%–64.09% and 46.96%–51.22% prediction accuracy on *Data_{old}* and *Data_{new}*, outperforming the most recent baseline EDITAS by 7.91%–19.88% and 5.86%–15.46%, respectively. Our analysis reveals that the observed improvements over previous baselines primarily stem from both our assertion generator and retriever, as demonstrated in Figures 3 and 4. The generator leverages extensive codebases to learn more meaningful vector representations for unit tests (e.g., the pre-training data of CodeT5 comprises 2.3 million functions from CodeSearchNet [31]), whereas baselines are trained only on a restricted corpus of test–assert pairs. Besides, the retriever identifies relevant assertions with both lexical and semantic matching, which is valuable for guiding the generator in generating correct assertions.

Answer to RQ3. Our comparison results demonstrate that: (1) RetriGen is general to different PLMs and sustainability achieves state-of-the-art performance, e.g., five involved PLMs achieve an accuracy of 54.29% on two datasets, outperforming the most recent EDITAS by 10.86% on average; (2) the default assertion generator (i.e., CodeT5) is natural and quite effective in the unit AG scenario, e.g., improving the accuracy of other PLMs by 7.81% on average across two datasets.

6 Discussion

Case Study. To further illustrate the retrieval and generation capabilities of RetriGen, we present two examples of assertions from real-world projects. Figure 3 illustrates a unique assertion from the OWLAPI project [3], which is only correctly generated by RetriGen, but all baselines fail to. In this example, IR_{ar} retrieves similar assertions based on lexical matching and returns an assertion within the same class as the input focal-test, i.e., “OWLontologyChangeRecord.” Although the retrieved assertion has a high token similarity with that of the input focal-test (e.g., both containing “org.semanticweb.owlapi.change.OWLontologyChangeRecord” and “record”), they are

Input Focal-Test	Focal-Test Retrieved by AG-RAG
<pre>//test prefix testGettersNotNull() { org.semanticweb.owlapi.change.OwlOntology ChangeRecord record = new org.semanticweb.owl api.change.OwlOntologyChangeRecord(mockOntolo gyID, mockChangeData); "<AssertPlaceHolder>"; } //focal method: getOntologyID() { return ontology.getOntologyID(); } //assertion (ground truth) org.junit.Assert.assertNotNull(record.getOnto logyID())</pre>	<pre>//test prefix testGettersNotNull() { org.semanticweb.owlapi.change.RemoveAxiom Data data = new org.semanticweb.owlapi.change RemoveAxiomData(mockAxiom); "" < AssertPlaceHolder > ""; } //focal Method: getAxiom() { return verifyNotNull(axiom); } //assertion org.junit.Assert.assertNotNull(data.getAxiom ())</pre>
Focal-Test Retrieved by Baselines <pre>//test prefix testEquals() { org.semanticweb.owlapi.change.OwlOntologyChangeRecord record1 = new org.semanticweb.owlapi.c hange.OwlOntologyChangeRecord(mockOntologyID, mockChangeData); org.semanticweb.owlapi.change.OwlOntologyChangeRecord record2 = new org.semanticweb.owlapi.c hange.o(mockOntologyID, mockChangeData);"<AssertPlaceHolder>";} //focal method: ... //assertion org.junit.Assert.assertEquals(record1, record2)</pre> <pre>IR_{ar}, RA_{adapt}^{NN}: org.junit.Assert.assertEquals(record1, record2)</pre> <pre>RA_{adapt}^H: org.junit.Assert.assertEquals(record, ontology)</pre> <pre>Intergation, ATLAS: org.junit.Assert.assertEquals(mockChangeData.ontology, record.getOntology ID())</pre> <pre>EDITAS: org.junit.Assert.assertNull(record.getOntologyID())</pre> <pre>RetriGen: org.junit.Assert.assertNotNull(record.getOntologyID())</pre>	

Fig. 3. Example of assertions generated by AG approaches from OWLAPI.

Input Focal-Test	Focal-Test by AG-RAG and Baselines
<pre>//test prefix test25() { byte[] expected = new byte[] { ((byte)(0)), ((byte)(0)) }; "<AssertPlaceHolder>"; } //focal method: build_filler(int) { return com.openddal.server.mysql.p roto.Proto.build_filler(len, ((byte) (0))); } //assertion (ground truth) org.junit.Assert.assertArrayEquals(exp ected, com.openddal.server.mysql.proto Proto.build_filler(2))</pre>	<pre>//test prefix test24() { byte[] expected = new byte[] { ((byte)(0)) }; "<AssertPlaceHolder>"; } //focal method: build_filler(int) { return com.openddal.server.mysql.p roto.Proto.build_filler(len, ((byte) (0))); } //assertion org.junit.Assert.assertArrayEquals(exp ected, com.openddal.server.mysql.proto Proto.build_filler(1))</pre>
<pre>IR_{ar}, RA_{adapt}^H, RA_{adapt}^{NN}, Intergation, EDITAS: org.junit.Assert.assertArrayEquals(expected, com.openddal.server.mysql.proto. Proto.build_filler(1))</pre> <pre>RetriGen : org.junit.Assert.assertArrayEquals(expected, com.openddal.server.mysql.proto. Proto.build_filler(2))</pre>	

Fig. 4. Example of assertions generated by AG approaches from OpenDDAL.

not responsible for testing similar functionalities. Thus, IR_{ar} and RA_{adapt}^{NN} directly return the wrong assertion. RA_{adapt}^H and EDITAS also fail to produce correct assertions as they make modifications to the wrong assertion type. For example, RA_{adapt}^H attempts to replace the second parameter within the assertion (“record2” → “ontology”). In contrast, RetriGen, which relies on lexical and semantic matching, accurately identifies a similar assertion from another class file. Despite significant

differences in lexical similarity, the two assertions share the same assertion type (i.e., `assertNotNull`) and parameter setting (i.e., `objectInstance.method()`). Thus, RetriGen is able to capture the edit patterns between the two focal-tests and performs the appropriate modifications on the retrieved assertion to generate the correct assertion.

Similarly, Figure 4 presents a real-world example from OpenDDAL [2], in which RetriGen and all baselines retrieve the same assertion for the given focal-test. The retrieved assertion is almost accurate, with only one parameter being corrected, as it targets the same focal-method (`"build_filler()"`) with the query input. However, existing approaches fail to assume this retrieved assertion is correct and directly return it as the final output. Benefiting from the code understanding capability of the utilized PLM-based generator, RetriGen successfully captures the semantic differences between the retrieved test prefix and the input test prefix (`"{{{(byte)(0)}}}"` $\rightarrow$ `"{{{(byte)(0)},(byte)(0)}}"`) and applies the corresponding edit operations (`"1"  $\rightarrow$  "2"`) to generate the correct assertion.

Potential of LLMs. As mentioned in Section 3, we consider encoder-decoder PLMs as foundation models of RetriGen and select CodeT5 because it is quite effective and the most popular PLM that is fine-tuned to support sequence-to-sequence code generation tasks. We notice that recent LLMs have been released with powerful performance, such as CodeLlama [41]. Most prior studies employ such LLMs in a zero-shot or few-shot setting, as fine-tuning these models with billions (or even more) of parameters is unaffordable due to device limitations [50]. In this section, we attempt to explore the preliminary potential of integrating LLMs with RetriGen. Due to device limitations, we select CodeLlama-7B as the foundation model to generate assertions without using retrieved assertions. The results demonstrate that CodeLlama-7B is able to achieve 71.42% accuracy, which is 11.44% better than RetriGen (64.09%). It is worth noting that the improvement is valuable as we do not perform the retrieval-augmented process, while our hybrid retriever can significantly enhance the prediction results, e.g., an improvement of 9.16% on the *Data_{old}* dataset in Table 4. Thus, we are confident that RetriGen is able to achieve better results when equipped with recent LLMs. While the potential of LLMs like CodeLlama is evident, their deployment raises concerns about computational efficiency in real-world applications. For example, fine-tuning billion-level LLMs may require access to high-performance GPU clusters, making the process unaffordable for many research and industrial teams. Thus, in the future, researchers can systematically analyze the tradeoffs between performance gains and computational costs when integrating larger LLMs into RetriGen. Besides, it is crucial to employ optimization strategies (parameter-efficient fine-tuning) to reduce computational overhead without sacrificing significant performance. Overall, the promising results motivate us to further explore the capabilities of RetriGen with newly released LLMs while balancing tradeoffs between effectiveness gains and efficiency.

Comparison with CEDAR. As mentioned in Section 4.3, following the methodology of the most recent work EDITAS [45], we select six prior AG approaches from three categories as baselines. To the best of our knowledge, this represents the largest set of baselines in the deep AG literature. Despite that, we notice that there may exist some studies leveraging LLMs in generating unit assertions [34]. For example, Nashid et al. [34] design a prompt-based few-shot learning strategy, CEDAR, which queries Codex for both AG and program repair. We exclude CEDAR as a baseline in Section 5 mainly due to the significant data leakage issue of the utilized black-box LLM Codex. In this section, we conduct an extended experiment to explore how RetriGen performs in comparison to CEDAR. However, we could not include CEDAR directly as a baseline because the datasets used in our study differ from those in theirs. CEDAR is evaluated on the *ATLAS* dataset, while our work and all baselines in Section 4.3 are evaluated on *Data_{old}* and *Data_{new}*. In particular, *Data_{old}* and *Data_{new}* are constructed based on *ATLAS*, where *Data_{old}* removes certain samples (e.g., those that could not be tokenized), and *Data_{new}* adds samples (e.g., those with unknown tokens). To address this, we reuse the reported results of CEDAR and execute RetriGen on the same *ATLAS*

dataset for a direct comparison. Given that CEDAR uses Codex, a 12-billion-parameter model, and Codex is not open-source, we chose CodeLlama-7B as the foundation model for generating assertions in RetriGen. The results demonstrate that, equipped with CodeLlama-7B, RetriGen is able to achieve 75.30% accuracy on the *ATLAS* dataset, comparable to CEDAR's 75.79% and 76.55% in two settings. It is important to note that CEDAR relies on six retrieved assertions to guide Codex in generating assertions, while we fine-tune CodeLlama-7B with only one assertion. Compared to PLMs, LLMs have the capability to handle longer inputs, allowing the number of retrieved examples to significantly enhance prediction results, which is also proven in the original paper of CEDAR, e.g., only 44.41% accuracy of CEDAR with the one-shot setting. Thus, we believe that RetriGen could achieve even better results with more retrieved assertions. In the future, we intend to undertake more comprehensive experiments with more retrieved assertions.

In fact, RetriGen and CEDAR are orthogonal, representing the cutting-edge approaches in model utilization strategies: fine-tuning and few-shot learning. In this work, we implement RetriGen in a fine-tuning manner instead of few-shot learning primarily due to concerns about data leakage. Particularly, few-shot typically relies on a highly powerful foundation model with a record-breaking number of parameters, such as ChatGPT and Codex. However, such commercial models are usually trained on publicly available data from the Internet to incorporate extensive domain knowledge to address various domain-specific issues (e.g., unit assertions), which may raise significant risks about data leakage. In contrast, the fine-tuning strategy enables us to choose much smaller open-source models that can be more effectively adapted to specific domains while mitigating data leakage concerns. For instance, in our work, RetriGen is implemented with CodeT5, a model with only 220 million parameters, which achieves impressive performance compared to the latest EditAS approach. Despite its smaller size (just 1.8% of Codex's parameters), CodeT5 effectively addresses data leakage concerns by relying entirely on open-source model weights and datasets. Besides, RetriGen is particularly valuable in real-world scenarios, such as where deployment resources are limited or where commercial models are unsuitable due to confidentiality concerns. This makes RetriGen a practical and reliable solution for tasks requiring domain-specific fine-tuning without compromising data security.

Potential of Fault Detection Capabilities. As mentioned in Section 4.4, we employ two metrics to evaluate the performance of RetriGen and baselines, which are widely adopted in prior deep AG studies [21]. In this section, we attempt to investigate the potential of generated assertions in uncovering real-world bugs. Following prior studies [11, 22, 29], we utilize EvoSuite [15] to generate test cases for Defects4J [23], which includes 835 bugs from 17 real-world Java projects. Given EvoSuite's reliance on randomized algorithms, we execute it 10 times per bug with different seeds to generate the final test cases. We exclude test cases that involve exception behavior, focusing specifically on AG. We then remove assertion statements generated by EvoSuite and apply RetriGen to predict the test assertions. To assess the performance of bug detection, we run the complete test cases on both buggy and fixed versions of all programs. A bug is considered detected if the test case fails on the buggy version but passes on the fixed one. We compare RetriGen with the best-performing baseline EDITAS due to dynamic execution overhead and computational resources. Both RetriGen and EDITAS are implemented using $Data_{new}$ as the training and retrieval corpus to ensure a fair comparison. Our results show that RetriGen and EDITAS detect 33 and 21 real-world bugs, respectively, with 20 and 8 unique detections. RetriGen surpasses EDITAS by 57.14% in total detected bugs and by 150% in unique detections. To our knowledge, this is the first attempt to evaluate the fault detection capabilities of deep assertion approaches [34, 45, 55, 59], highlighting RetriGen's potential in detecting bugs. In the future, we recommend that researchers conduct more extensive studies with real-world bugs to evaluate deep AG approaches.

Practical Deployment. As shown in Figure 1, RetriGen is designed to predict appropriate assertion statements when provided with focal methods and test prefixes. In practice, RetriGen can be deployed as a complement to automated test case generation tools or as a code completion plugin for developers. First, existing automated test generation tools (e.g., EvoSuite) are skilled at generating test prefixes with high code coverage but often struggle to understand the semantics of focal methods and generate meaningful assertions. RetriGen can address this gap by accepting a test prefix generated by existing tools along with its focal method as inputs and generating appropriate assert statements. In this context, RetriGen is not a replacement for existing automated test generation tools but a complementary technique that enhances their effectiveness. Our preliminary results on Defects4J demonstrate that when provided with focal methods and the test prefixes generated by EvoSuite, AG-RAG can generate meaningful assertions capable of detecting real-world bugs. Second, RetriGen can be integrated into an IDE as a code completion plugin for developers, offering suggested assertion statements while they write test code. For instance, when a developer needs to validate the correctness of software units, they begin by writing a test prefix, i.e., a sequence of call statements that invoke the specific behavior of the unit under test. RetriGen then assists by automatically generating the necessary test assertions based on the context of the focal method and the test prefix. This plugin is particularly valuable because manually crafting effective assert statements requires a deep understanding of the program's functionality and specifications. Our experimental results on *Data_{old}* demonstrate that RetriGen predicts 64.09% of correct assertions when given the focal method under test and a developer-written test prefix. This result highlights the potential of RetriGen in automated code completion scenarios, where developers can easily select appropriate assert statements recommended by RetriGen, significantly reducing the manual effort involved in writing assertions.

7 Threats to Validity

The first threat to validity pertains to the evaluation metrics. In this work, we strictly follow the evaluation protocols established in prior work [45, 55, 59] and employ prediction accuracy to measure the effectiveness of the generated assertion. However, we are unable to execute generated assertions due to the limitations of the utilized datasets (only containing focal and test methods) [30], making it impossible to employ dynamic metrics such as defect detection rate. To mitigate this threat, we utilize Defects4J to assess whether the generated assertions can detect real-world bugs. Besides, since all baselines omit efficiency metrics, such as training or inference times in their evaluation, we cannot provide a direct comparison with baselines in this regard. To mitigate this threat, we include a code-aware metric, CodeBLEU, that has not yet been adopted in prior AG work. In the future, we will conduct an extensive evaluation of such deep AG approaches in terms of computational costs.

The second threat to validity arises from the possibility of data leakage in PLMs. PLMs are usually pre-trained with a mass of open-source projects, which may contain the test methods from the two AG datasets. To address the concern, we query the pre-training datasets (e.g., CodeSearchNet [31]) of studied PLMs and find such PLMs do not have access to any test cases (including assertions) during pre-training. It is worth noting that the data leakage concern motivates our choice of open-source PLMs (mentioned in Section 5.3) instead of black-box PLMs or LLMs. Therefore, we believe that this concern does not significantly impact our conclusions.

The third threat to validity is the potential limitation of our findings in generalizing to other benchmarks, programming languages, and PLMs. First, the two benchmarks may not fully capture the diversity of assertion patterns or coding styles across various domains. To mitigate the threat, we also utilize Defects4J to evaluate the fault detection capabilities of RetriGen in a more realistic assessment setting. Second, we only evaluate RetriGen on Java programs. However, we believe

that the impact of this threat is relatively minor because (1) Java is the most targeted language in the AG field [21]; (2) adopted datasets are widely utilized and sufficiently large-scale to yield reliable conclusions; and (3) RetriGen is language-agnostic to support multiple languages naturally. Third, we implement RetriGen with the recent PLM CodeT5. To mitigate the threat, we select four other PLMs from three types of model architectures. Considering that selected PLMs cover different architectures, organizations, parameter sizes, and pre-training datasets, we are confident in extending our findings to newly released larger LLMs. In the future, we attempt to explore the performance of RetriGen with more benchmarks, programming languages and PLMs.

8 Conclusion and Future Work

In this work, we present RetriGen, a novel retrieval-augmented AG approach that leverages the advances of external codebases and PLMs. RetriGen first builds a hybrid retriever to search the most relevant assertion for a query focal-test from external codebases by both the lexical and semantic similarity. RetriGen then utilizes off-the-shelf CodeT5 as the assertion generator to predict assertions fine-tuned with both focal-tests and additional retrieved assertions. The experimental results on two datasets show the superior performance of RetriGen against all six baselines on two metrics, e.g., achieving 57.66% and 73.24% in terms of accuracy and CodeBLE, outperforming all state-of-the-art AG techniques by 50.66% and 14.14% on average. We also demonstrate that RetriGen is able to generate a mass of unique assertions that all baselines fail to generate, e.g., 1,598 and 1,818 ones on two datasets, 3.71X and 4.58X more than the most recent technique EDITAS. In the future, we plan to explore the applicability of RetriGen with larger PLMs, other programming languages, and datasets.

Acknowledgments

The authors would like to thank the editors and anonymous reviewers for their time and comments.

References

- [1] Hugging Face. 2024. Retrieved from <https://huggingface.co/>
- [2] OpenDDAL. 2024. Retrieved from <https://github.com/neradb/openddal>
- [3] OWLAPI. 2024. Retrieved from <https://github.com/owlcs/owlapi>
- [4] PyTorch. 2024. Retrieved from <https://pytorch.org/>
- [5] M. Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track*, 263–272.
- [6] Andrea Arcuri. 2019. RESTful API automated test case generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology* 28, 1 (2019), 1–37.
- [7] Earl T. Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. 2014. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 306–317.
- [8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A framework for LLM-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 572–576.
- [9] Ermira Daka and Gordon Fraser. 2014. A survey on unit testing practices and problems. In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering*, 201–211.
- [10] Tung Dao, Lingming Zhang, and Na Meng. 2017. How does execution information help with information-retrieval based bug localization? In *Proceedings of the 2017 IEEE/ACM 25th International Conference on Program Comprehension*, 241–250.
- [11] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*, 2130–2141.
- [12] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. 2002. Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering* 28, 2 (2002), 159–182.

- [13] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering*. IEEE, 31–53.
- [14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547.
- [15] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ACM, 416–419.
- [16] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 935–947.
- [17] Siqi Gu, Chunrong Fang, Quanjin Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. TestART: Improving LLM-based unit test via co-evolution of automated generation and repair iteration. arXiv:2408.03095. Retrieved from <https://arxiv.org/abs/2408.03095>
- [18] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 7212–7225.
- [19] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the 9th International Conference on Learning Representations*, 1–18.
- [20] Alan Hartman. 2002. Is ISTTA research relevant to industry? *ACM SIGSOFT Software Engineering Notes* 27, 4 (2002), 205–206.
- [21] Yibo He, Jiaming Huang, Hao Yu, and Tao Xie. 2024. An empirical study on focal methods in deep-learning-based approaches for assertion generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1750–1771.
- [22] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian Elbaum, and Willem Visser. 2023. Neural-based test oracle generation: A large-scale evaluation and lessons learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 120–132.
- [23] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis*, 437–440.
- [24] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen Tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP '20)*. Association for Computational Linguistics, 6769–6781.
- [25] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th International Conference on Software Engineering*, 919–931.
- [26] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, Vol. 33, 9459–9474.
- [27] Jia Li, Yongmin Li, Ge Li, Xing Hu, Xin Xia, and Zhi Jin. 2021. EDITSUM: A retrieve-and-edit framework for source code summarization. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering*, 155–166.
- [28] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. 2022. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1035–1047.
- [29] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards more realistic evaluation for neural test oracle generation. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 589–600.
- [30] Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding build issue resolution in practice: Symptoms and fix patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 617–628.
- [31] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1–16.

- [32] Antonio Mastropaolo, Nathan Cooper, David Nader Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2022. Using transfer learning for code-related tasks. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1580–1598.
- [33] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering*. IEEE, 336–347.
- [34] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering*. IEEE, 2450–2462.
- [35] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-directed random testing for java. In *Companion to the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 815–816.
- [36] Rongqi Pan, Taher Ahmed Ghaleb, and Lionel C. Briand. 2024. LTM: Scalable and black-box similarity-based test suite minimization based on language models. *IEEE Transactions on Software Engineering* 50, 11 (2024), 3053–3070.
- [37] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2719–2734.
- [38] Qianyang Peng, August Shi, and Lingming Zhang. 2020. Empirically revisiting and enhancing IR-based test-case prioritization. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 324–336.
- [39] Strategic Planning. 2002. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. National Institute of Standards and Technology, 1.
- [40] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
- [41] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [42] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage guided test generation in regression setting using LLM. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.
- [43] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.
- [44] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A reproducible benchmark of recent Java bugs. In *Proceedings of the 2024 IEEE/ACM 21st International Conference on Mining Software Repositories*, 118–122.
- [45] Weifeng Sun, Hongyan Li, Meng Yan, Yan Lei, and Hongyu Zhang. 2023. Revisiting and improving retrieval-augmented deep assertion generation. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering*, 1123–1135.
- [46] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1340–1359.
- [47] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics*, 4173–4198.
- [48] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*, 54–64.
- [49] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th International Conference on Software Engineering*, 2291–2302.
- [50] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 04 (2024), 911–936.
- [51] Weishi Wang, Yue Wang, Shafiq Joty, and Steven C. H. Hoi. 2023. RAP-Gen: Retrieval-augmented patch generation with CodeT5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 146–158.
- [52] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 8696–8708.
- [53] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based unit test generation via method slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1258–1268.

- [54] Cody Watson, Nathan Cooper, David Nader Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A systematic literature review on the use of deep learning in software engineering research. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–58.
- [55] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 1398–1409.
- [56] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering*, 1482–1494.
- [57] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1607–1619.
- [58] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A survey on deep learning for software engineering. *ACM Computing Surveys* 54, 10s (2022), 1–73.
- [59] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated assertion generation via information retrieval and its integration with deep learning. In *Proceedings of the 44th International Conference on Software Engineering*, 163–174.
- [60] Wei Yuan, Quanjun Zhang, Tiek He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: Continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 678–690.
- [61] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving ChatGPT for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.
- [62] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2024. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2024), 55:1–55:69.
- [63] Quanjun Zhang, Chunrong Fang, Yang Xie, Yuxiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. 2024. A systematic literature review on large language models for automated program repair. arXiv:2405.01466. Retrieved from <https://arxiv.org/abs/2405.01466>
- [64] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A survey on large language models for software engineering. arXiv:2312.15223. Retrieved from <https://arxiv.org/abs/2405.01466>
- [65] Quanjun Zhang, Chunrong Fang, Bowen Yu, Weisong Sun, Tongke Zhang, and Zhenyu Chen. 2024. Pre-trained model-based automated software vulnerability repair: How far are we? *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 2507–2525.
- [66] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 535–547.
- [67] Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2024. Exploring automated assertion generation via large language models. *ACM Transactions on Software Engineering and Methodology* 34, 3 (Oct. 2024), 1–25. DOI: <https://doi.org/10.1145/3699598>
- [68] Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. A critical review of large language model on software engineering: An example from ChatGPT and automated program repair. arXiv:2310.08879. Retrieved from <https://arxiv.org/abs/2310.08879>
- [69] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 872–872.
- [70] Qihao Zhu, Qingyuan Liang, Zeyu Sun, Yingfei Xiong, Lu Zhang, and Shengyu Cheng. 2024. GrammarT5: Grammar-integrated pretrained encoder-decoder neural model for code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–13.

Received 3 October 2024; revised 30 December 2024; accepted 14 February 2025